

Experiment – 3

Name: Aryan

UID: 24BAI70179

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: /2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 3.1

Aim:

To design and implement a secure authentication system using JWT for user login and session management.

Objectives:

- To understand authentication mechanisms in web applications.
- To implement token-based authentication using JWT.
- To manage user sessions in a stateless architecture.
- To handle token storage and validation securely.

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Frontend: React.js
- Backend: Node.js with Express.js
- Database: MongoDB / MySQL (for storing user credentials)
- Authentication Library: jsonwebtoken, bcrypt.js
- Code Editor: Visual Studio Code
- Package Manager: npm
- API Testing Tool: Postman
- Browser: Google Chrome / Microsoft Edge

Theory:

JSON Web Token (JWT) is a compact, self-contained, and secure way of transmitting information between a client and a server as a JSON object. It is widely used to implement stateless authentication, where the server does not need to store session information in memory or a database.

A JWT consists of three parts, separated by dots: Header, Payload, and Signature. The Header specifies the token type and signing algorithm. The Payload contains claims such as user ID, role, and expiry time. The Signature is generated by encoding the header and payload and signing them with a secret key, ensuring the token has not been tampered with.

In a typical JWT authentication flow, the user submits login credentials to the server. The server validates the credentials and, if correct, generates a signed JWT and returns it to the client. The client stores this token and includes it in the Authorization header of subsequent requests. The server verifies the token's signature and expiry on each request before granting access to protected resources.

Unlike traditional session-based authentication, JWT-based authentication does not require the server to maintain session state, making it well suited for scalable and distributed systems. However, secure storage of the token on the client (for example, using httpOnly cookies rather than localStorage) and proper handling of token expiry and refresh are essential to prevent security vulnerabilities such as token theft.

Implementation:

1. Set up a Node.js project and install Express.js.
2. Install the required dependencies: jsonwebtoken, bcrypt, and mongoose.
3. Create a User schema/model with a securely hashed password field.
4. Implement a registration API that hashes the password before saving the user.
5. Implement a login API that verifies credentials and issues a signed JWT.
6. Create a middleware function to verify the JWT on protected routes.
7. Store the JWT securely on the client after successful login.
8. Implement token expiry and refresh-token logic.
9. Test the registration, login, and protected-route flow using Postman.
10. Integrate the authentication APIs with a React frontend login form.

Code:**• authController.js**

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const User = require('../models/User');

exports.register = async (req, res) => {
  const { name, email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email, password: hashedPassword });
  res.status(201).json({ message: 'User registered successfully', id: user._id });
};

exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(404).json({ message: 'User not found' });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );

  res.json({ message: 'Login successful', token });
};
```

• authMiddleware.js

```
const jwt = require('jsonwebtoken');

function verifyToken(req, res, next) {
  const authHeader = req.headers['authorization'];
```

```
const token = authHeader && authHeader.split(' ')[1];
if (!token) return res.status(401).json({ message: 'No token provided' });

jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
  if (err) return res.status(403).json({ message: 'Invalid or expired token' });
  req.user = decoded;
  next();
});
}

module.exports = verifyToken;
```

• Login.jsx (React)

```
import { useState } from 'react';
import axios from 'axios';

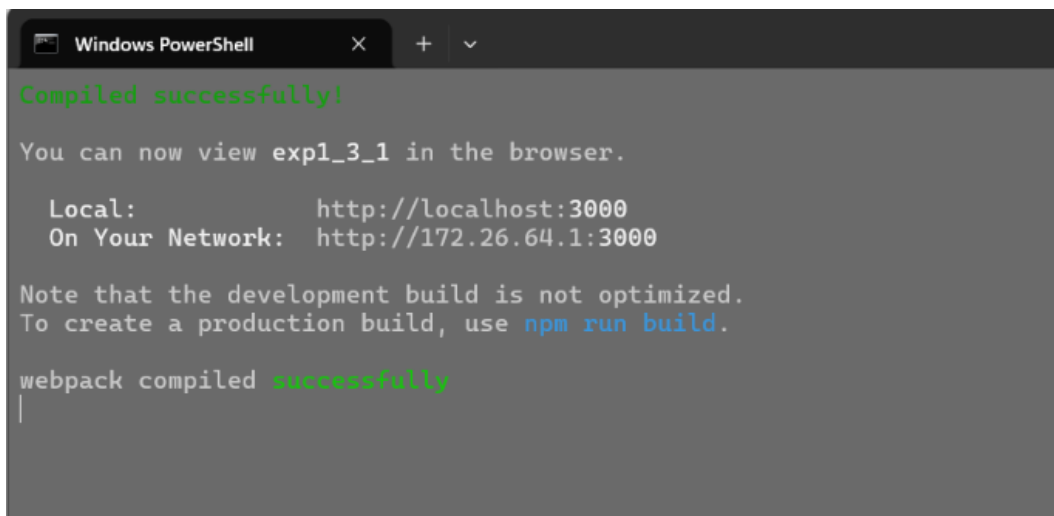
function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleLogin = async (e) => {
    e.preventDefault();
    const res = await axios.post('http://localhost:5000/api/auth/login', {
      email, password,
    });
    localStorage.setItem('token', res.data.token);
  };

  return (
    <form onSubmit={handleLogin}>
      <input value={email} onChange={(e) => setEmail(e.target.value)} placeholder="Email" />
      <input type="password" value={password} onChange={(e) => setPassword(e.target.value)}
placeholder="Password" />
      <button type="submit">Login</button>
    </form>
  );
}

export default Login;
```

Output:



```
Windows PowerShell
Compiled successfully!

You can now view exp1_3_1 in the browser.

Local:      http://localhost:3000
On Your Network:  http://172.26.64.1:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

Fig A: JWT Login – Token Response in Postman

Welcome, aryan

Signed in as **User** — 24bai70179@cuchd.in

My Profile

View your account details.

Fig B: React Login Page

Learning Outcomes:

- Understand authentication mechanisms used in modern web applications.
- Implement token-based authentication using JWT.
- Design stateless session management for scalable applications.
- Securely hash and validate user passwords.
- Create middleware to protect backend routes.
- Integrate a JWT-secured API with a React frontend.

Conclusion:

A secure, stateless authentication system was successfully designed and implemented using JSON Web Tokens. User credentials were validated and securely hashed, tokens were issued upon successful login, and protected routes were secured using middleware that verifies token validity. This approach provides a scalable foundation for managing user sessions without server-side session storage.

EXPERIMENT – 3.2

Aim:

To implement role-based access control and secure application routes based on user permissions.

Objectives:

- To understand authorization mechanisms in applications.
- To implement RBAC for access control.
- To protect routes based on user roles.
- To dynamically render UI elements based on permissions.

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Frontend: React.js
- Backend: Node.js with Express.js
- Database: MongoDB / MySQL
- Authentication Library: jsonwebtoken
- Code Editor: Visual Studio Code
- Package Manager: npm
- API Testing Tool: Postman
- Browser: Google Chrome / Microsoft Edge

Theory:

Authentication verifies who a user is, while authorization determines what an authenticated user is allowed to do. Role-Based Access Control (RBAC) is a widely used authorization model in which permissions are assigned to roles, and roles are assigned to users, rather than granting permissions to individual users directly.

In a typical RBAC implementation, each user account is associated with one or more roles, such as Admin, Editor, or Viewer. When a user logs in, their role is embedded in the JWT payload. On each request to a protected route, a middleware function checks the role encoded in the token against the roles permitted to access that route, rejecting the request if the user's role does not have sufficient permissions.

On the frontend, RBAC is also used to conditionally render UI elements, such as showing an admin dashboard link only to users with the Admin role. This improves both security and user experience by preventing unauthorized users from seeing or attempting actions they are not permitted to perform.

Implementation:

1. Define the application's roles, such as Admin, Editor, and Viewer.
2. Add a role field to the User schema.
3. Embed the user's role inside the JWT payload during login.
4. Create a role-based authorization middleware on the backend.
5. Protect specific routes by allowing only certain roles to access them.
6. Decode the JWT on the frontend to identify the logged-in user's role.
7. Conditionally render UI components and navigation links based on role.

8. Test the application using accounts with different roles to verify access control.

Code:

- **roleMiddleware.js**

```
function authorizeRoles(...allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Access denied: insufficient permissions' });
    }
    next();
  };
}

module.exports = authorizeRoles;
```

- **adminRoutes.js**

```
const express = require('express');
const router = express.Router();
const verifyToken = require('../middleware/authMiddleware');
const authorizeRoles = require('../middleware/roleMiddleware');

router.get(
  '/dashboard',
  verifyToken,
  authorizeRoles('Admin'),
  (req, res) => {
    res.json({ message: 'Welcome to the Admin dashboard' });
  }
);

module.exports = router;
```

- **ProtectedLink.jsx (React)**

```
import { jwtDecode } from 'jwt-decode';

function Navbar() {
  const token = localStorage.getItem('token');
  const role = token ? jwtDecode(token).role : null;

  return (
    <nav>
      <a href="/">Home</a>
      {role === 'Admin' && <a href="/admin">Admin Dashboard</a>}
      {(role === 'Admin' || role === 'Editor') && <a href="/posts/new">New Post</a>}
    </nav>
  );
}

export default Navbar;
```

Output:

Sign In

Log in to access your dashboard

Email

Password

Sign In

Quick-fill a demo account:

Admin

Employee

User

No account? [Register here](#)

Fig A: Access Denied Response for Unauthorized Role

Welcome, aryan

Signed in as **User** — 24bai70179@cuchd.in

My Profile

View your account details.

Fig B: Admin Dashboard Link Visible Only to Admin Role

Learning Outcomes:

- Understand the difference between authentication and authorization.
- Implement Role-Based Access Control in a full-stack application.
- Protect backend routes based on user roles.
- Embed and decode role information within a JWT.
- Dynamically render frontend UI elements based on user permissions.
- Test and verify access control across multiple user roles.

Conclusion:

Role-based access control was successfully implemented to secure application routes and resources according to user permissions. By embedding role information in the JWT and validating it through dedicated middleware, unauthorized access to protected resources was effectively prevented. The frontend was also updated to dynamically render UI elements based on the logged-in user's role, resulting in a more secure and user-appropriate application experience.